

大規模リファクタリングで学ぶ 用語ま とめ(第 5 章)

～ レガシーコードの整え方から、設計の型・注文/決済・性能改善まで ～

この文書は、数十ファイルにまたがる大規模なリファクタリングでよく出てくる言葉を、身近な例えとセットで分かるようにまとめたものです。

作成日:2026 年 6 月 18 日

はじめに — この文書の使い方

この文書は、用語の意味を初学者向けにやさしくまとめたものです。むずかしい言葉は、レストランや建物などの身近なものにたとえて説明しています。

各用語は「ひとことで言うと → 身近な例え → 具体例 → つながり」の同じ型で並べています。まずは「ひとことで言うと」だけ拾い読みしても大丈夫です。

◆ この文書で扱う用語(テーマ別)

- **リファクタを始める前に:** レガシーコード / リファクタ / テストカバレッジ / デグレ / スプリント
- **安全に進める道具立て:** Subagent(Task tool) / git worktree / diff / hotfix / コンパイル
- **コードを整える設計:** メソッドの責務分離 / シグネチャ / DTO クラス / スコープ / アクセス修飾子
- **コードの中の実物(注文・決済):** ORDER_STATUS / enum / findByCustomerId / PaymentService
- **性能を引き上げる:** マルチスレッド化 / ボトルネック

第1部 — リファクタを始める前に

1. レガシーコード(Legacy Code)

ひとことで言うと:長年動いてきたものの、作りが古く手を入れづらくなった昔のコードのこと。

増築をくり返して間取りが入り組んだ、古い建物のようなものです。今も使えてはいますが、どの部屋がどこに繋がっているか分かりにくく、うかつに壊すと別の場所に不具合が出ます。

◆ レガシーコードにありがちなこと

観点	レガシーなコード	整理されたコード
見通し	どこで何をしているか追いにくい	役割ごとに分かれて読みやすい
テスト	ほとんど無く、直すのが怖い	テストがあり安心して直せる
変更	一箇所直すと別が壊れがち	影響範囲がはっきりしている

【つながり】 この古いコードを、動きを変えずに整え直していく作業が、次の「リファクタ」です。

2. リファクタ(リファクタリング / Refactoring)

ひとことで言うと:プログラムの動きを変えずに、中身の作りだけをきれいに整え直すことのこと。

散らかった部屋を、物の場所を決めて片づけ直すようなものです。置いてある物(機能)は何も捨てず増やさず、取り出しやすいように引き出しを整理するイメージです。

◆ リファクタでやること

- 長すぎる処理を小さく分ける
- 分かりにくい名前を分かりやすく直す
- 重複したコードを一つにまとめる

【つながり】動きを変えていないと安心して進めるために、まず足場として用意するのが、次の「テストカバレッジ」です。

3. テストカバレッジ(Test Coverage)

ひとことで言うと:コードのうち、どれだけの範囲がテストで確かめられているかを示す割合のこと。

家の掃除で「部屋全体のうち何割を拭けたか」を表す進み具合のようなものです。カバレッジが高いほど、見ていない隅(未確認のコード)が少なく、安心して模様替え(リファクタ)できます。

◆ カバレッジが高いと

- 直したとき、壊れた箇所にすぐ気づける
- テストが通れば動きを変えていない自信になる
- どこが未確認かが数字で見える

【つながり】カバレッジが低いまま直して、いつのまにか機能を壊してしまう失敗が、次の「デグレ」です。

4. デグレ(デグレード / Regression)

ひとことで言うと:修正のせいで、今まで動いていた機能が逆に壊れてしまうことのこと。

棚の奥を片づけたら、手前に並べてあった物を倒してしまうようなものです。よかれと思った変更が、別の場所の動きをこっそり後退させてしまう、リファクタで一番怖い失敗です。

◆ デグレを防ぐには

- 直す前にテストで現状の動きを固めておく
- 直したあと全テストを通して確認する
- 変更は小さく区切って進める

【つながり】こうした作業を「いつまでに、どれだけやるか」区切って計画する単位が、次の「スプリント」です。

5. スプリント(Sprint)

ひとことと言うと:短い期間を区切って、その中で決めた分の開発を進めるひと区切りのこと。

学校の「一学期」のように区切られた、短い計画期間のようなものです。たとえば2週間で1スプリントと決め、その間にやることを絞って取り組み、終わりに振り返ります。

◆ スプリントの流れ

- 最初に、この期間でやることを決める
- 期間中はそれに集中して進める
- 終わりに成果と進め方を振り返る

【つながり】 決めた作業を一人で抱えず、AIに調べ物や下調べを任せる助っ人が、次の「Subagent(Task tool)」です。

第 2 部 — 安全に進める道具立て

6. Subagent(サブエージェント / Task tool)

ひとことと言うと: Claude Code が、調べ物や下作業を任せるために別働で動かす分身の担当のこと。

レストランで料理長が、仕込みを別の調理スタッフに任せるようなものです。本体が指示を出し、サブエージェントが「この部分を調べておいて」といった作業を厨房の裏で並行して進めます。

◆ Subagent が役立つ場面

- 大量のファイルを手分けして調べる
- 本体の会話を散らかさず下調べを任せる
- 似た作業をまとめて並行で進める

【つながり】 分身たちが互いの作業を踏まないように、作業場所そのものを分ける仕組みが、次の「git worktree」です。

7. git worktree(ギット ワークツリー)

ひとことと言うと: 同じプロジェクトの作業フォルダを複数に分けて、別々の作業を同時に進められる仕組みのこと。

一軒の家(リポジトリ)に作業部屋を増設して、人ごとに別室で作業するようなものです。机(フォルダ)を分けるので、隣の作業中の書きかけを汚さずに、別の改修を同時に進められます。

◆ 使い方(イメージ)

```
# 「refactor」用の作業部屋を別フォルダに用意する
git worktree add ../work-refactor refactor
# → 元のフォルダはそのまま、増設先で別作業ができる
```

【つながり】 別々の部屋で進めた変更が「どこをどう変えたか」を見比べるのが、次の「diff」です。

8. diff(ディフ / 差分)

ひとことと言うと:変更の前と後を見比べて、どこが変わったかだけを抜き出して示したもののこと。

間違い探しで、2枚の絵の「違う箇所」だけに丸を付けたもののようなものです。ファイル全体ではなく、消した行と足した行だけが並ぶので、何をしたのか一目で分かります。

◆ diff の見え方(イメージ)

```
int price = item.getPrice();  
- int total = price;           ← 消した行  
+ int total = price * quantity; ← 足した行
```

【つながり】 差分を確認して問題が見つかり、急いで本番を直す緊急の修正が、次の「hotfix」です。

9. hotfix(ホットフィックス / 緊急修正)

ひとことと言うと:本番で起きている問題を、後回しにせず最優先で当てる緊急の修正のこと。

水道管が急に漏れたとき、まず応急処置のテープを巻く対応のようなものです。じっくりした改修(リファクタ)とは別枠で、今困っている事態だけを最短で止めにいきます。

◆ hotfix の進め方

- 原因の中でも「今すぐ困る部分」だけ直す
- 影響を最小限にして素早く本番へ反映する
- 落ち着いてから根本対応をあらためて行う

【つながり】 修正を反映する前に「文法として正しく組み上がるか」を確かめる工程が、次の「コンパイル」です。

10. コンパイル(Compile)

ひとことと言うと:人が書いたコードを、コンピュータが実行できる形へ翻訳して組み上げる工程のこと。

レシピ(コード)を、実際に作れる手順書へ翻訳して通すようなものです。書き間違い(文法ミス)があるところで止まって教えてくれるので、動かす前の最初の関所になります。

◆ コンパイルでよく分かること

- 綴りや記号の書き間違い(文法エラー)
- 存在しない名前を呼んでいないか
- 型(データの種類)が食い違っていないか

【つながり】 翻訳を通すために、長すぎる処理を意味のかたまりへ切り分ける考え方が、次の「メソッドの責務分離」です。

第3部 — コードを整える設計

11. メソッドの責務分離(Separation of Responsibilities)

ひとことと言うと:一つの処理にあれこれ詰め込まず、役割ごとに小さな処理へ分けることのこと。

一人で接客も調理も会計もこなす店員を、役割ごとの担当に分けるようなものです。一つのメソッドが何でも屋になっていると読みにくいので、「これだけをする」単位に切り分けます。

◆ 分けるとうれしいこと

- 一つひとつが短く読みやすくなる
- 直す範囲が小さく、デグレしにくい
- 同じ処理を別の場所から使い回せる

【つながり】 処理を分けると変わってくる「メソッドの呼び出しの形」が、次の「シグネチャ」です。

12. シグネチャ(Signature / 署名)

ひとことと言うと:メソッドの名前・受け取る引数・返す型をまとめた、呼び出し方を表す形のこと。

自動販売機の「投入口の形と出てくる商品」の決まりのようなものです。何を入れれば(引数)何が出るか(戻り値)が決まっているので、使う側はその形さえ守れば呼び出せます。

◆ シグネチャの読み方(イメージ)

```
// 名前: calculateTotal  
// 受け取る: Order(注文) / 返す: int(合計金額)  
int calculateTotal(Order order)
```

【つながり】 引数や戻り値として、データをひとまとめにして受け渡す入れ物が、次の「DTOクラス」です。

13. DTO クラス(Data Transfer Object)

ひとことと言うと:複数のデータをひとまとめにして受け渡すためだけの、入れ物役のクラスのこと。

お弁当箱のように、おかず(データ)を仕切って一緒に運ぶ容器のようなものです。名前・住所・金額などをバラバラに渡す代わりに、一つの箱に詰めてまとめて持ち運びます。

◆ DTO の書き方(イメージ)

```
// 注文の表示用データを、まとめて運ぶ入れ物
class OrderDto {
    String customerName; // 客の名前
    int totalPrice;      // 合計金額
}
```

【つながり】 この入れ物の中身が「どこからどこまで見えてよいか」という有効範囲が、次の「スコープ」です。

14. スコープ(Scope / 有効範囲)

ひとことと言うと:変数や名前が「どこから見えて、どこまで使えるか」という通用する範囲のこと。

建物の中で「この部屋でだけ使える」社員証のようなものです。部屋(カッコ)の中で作ったものはその部屋の中だけで通用し、外へ出ると使えなくなります。

◆ スコープのイメージ

```
void calc() {
    int x = 10; // x はこの部屋(波カッコ)の中だけ有効
}
// ここでは x は見えない(使えない)
```

【つながり】 この見える範囲を、部屋単位ではなくクラスの外向けに決めるのが、次の「アクセス修飾子」です。

15. アクセス修飾子(Access Modifier)

ひとことと言うと:そのデータや処理を「誰まで使ってよいか」を決めるための指定のこと。

部屋ごとの「関係者以外立入禁止」の張り紙のようなものです。誰でも入れる入口(public)か、中の人だけの部屋(private)かを決めて、勝手な持ち出しを防ぎます。

◆ 代表的な指定

指定	意味	たとえ
public	どこからでも使ってよい	誰でも入れる正面入口
private	そのクラスの中だけ	関係者だけの部屋
protected	仲間(継承先)まで	社員と関連会社まで

【つながり】 こうした設計の決まりが実際に使われている注文コードの一例が、次の「ORDER_STATUS」です。

第 4 部 — コードの中の実物(注文・決済)

16. ORDER_STATUS(オーダー ステータス)

ひとことと言うと:注文が今どの段階にあるか(受付・発送済みなど)を表す状態の値のこと。

宅配便の追跡表示「受付 → 配送中 → お届け済み」のようなものです。注文という一つの荷物が、今どの状態にいるのかを一目で分かるように名前を付けて管理します。

◆ ORDER_STATUS の値の例

```
NEW          // 受付したばかり
PAID         // 支払い済み
SHIPPED      // 発送済み
DELIVERED    // お届け済み
```

【つながり】 こうした「決まった選択肢」を、間違えにくい形でコードに表す仕組みが、次の「enum」です。

17. enum(イナム / 列挙型)

ひとことと言うと:あらかじめ決めた選択肢だけを値として持てるようにした、専用のデータの種類のこと。

信号機の「赤・黄・青」だけしか選べないようなものです。文字で自由に書くと打ち間違いが起きますが、enum なら決めた候補からしか選べず、ありえない値を防げます。

◆ enum の書き方(イメージ)

```
// 注文の状態を、決めた選択肢だけに限定する
enum OrderStatus {
    NEW, PAID, SHIPPED, DELIVERED
}
```

【つながり】 この状態を手がかりに、特定の客の注文を探し出すメソッドが、次の「findByCustomerId」です。

18. findByCustomerId(ファインド バイ カスタマー アイディー)

ひとことで言うと:お客様の ID をもとに、その人の注文データを探して取り出すメソッドのこと。

電話帳で「名前」から相手の連絡先を引くようなものです。顧客 ID(目印)を渡すと、その人に紐づく注文だけを倉庫(データベース)から選んで返してくれます。

◆ 名前の読み解き方(イメージ)

```
// find(探す) + By(~で) + CustomerId(顧客 ID)
// = 「顧客 ID で探す」という意味の名前
List<Order> findByCustomerId(Long customerId)
```

【つながり】 探し出した注文に対して、支払いの処理を受け持つ専門の担当が、次の「PaymentService」です。

19. PaymentService(ペイメント サービス)

ひとことで言うと:注文の支払いに関わる処理(決済・確認・記録)をまとめて受け持つ担当クラスのこと。

レストランの会計レジ係のようなものです。料理(注文)が決まったあと、金額を確かめてお金を受け取り、領収の記録を残すところまでを一手に引き受けます。

◆ PaymentService がすること

- 支払い金額が正しいか確かめる
- 決済の処理を実行する
- 支払いの結果を記録に残す

【つながり】 支払いのように時間のかかる処理を、同時並行で速くさばく工夫が、次の「マルチスレッド化」です。

第5部 — 性能を引き上げる

20. マルチスレッド化(Multi-threading)

ひとことで言うと:複数の処理を同時並行で走らせて、全体を速く片づけられるようにすることのこと。

スーパーのレジを1台から複数台に増やして、客を同時にさばく仕組みのようなものです。一列に並んで順番待ちする代わりに、いくつもの窓口で並行して進めるので、待ち時間が短くなります。

◆ マルチスレッド化の注意点

- 同時に進める分、速くなることが多い
- 同じデータを取り合うと不具合が起きやすい
- 増やしすぎると逆に管理が重くなる

【つながり】 同時並行にしても全体が思うほど速くならないとき、足を引っばる箇所が、次の「ボトルネック」です。

21. ボトルネック(Bottleneck)

ひとことで言うと:全体の速さを一番制限している、流れの中の詰まった箇所のこと。

瓶(ボトル)の細い首のように、そこだけ細くて流れが滞る場所のようなものです。レストランで席が空いていても厨房が一つなら料理は遅いまま、というように、一番遅い所が全体の速さを決めます。

◆ ボトルネック対策の流れ

- どこが一番遅いかを測って特定する
- そこを集中的に直して流れを広げる
- 直したら、次に遅い所をまた探す

【つながり】 詰まりを一つずつ解消する積み重ねが、古いコードを使いやすく直す力になり、最初の「レガシーコード」の解消へと還っていきます。

付録 A — 全体まとめ表

用語	ひとことと言うと	身近な例え
レガシーコード	古くて手を入れづらい昔のコード	増築で入り組んだ古い建物
リファクタ	動きを変えず中身を整え直す	物の場所を決めて片づけ直す
テストカバレッジ	テストで確かめた範囲の割合	部屋の何割を拭けたかの進み具合
デグレ	修正で動いていた機能が壊れる	奥を片づけて手前の物を倒す
スプリント	期間を区切る開発のひと区切り	学校の一学期のような計画期間
Subagent(Task tool)	下作業を任せる別働の分身	仕込みを任せる調理スタッフ
git worktree	作業フォルダを分けて同時進行	家に作業部屋を増設して別室作業
diff	変わった所だけ抜き出す差分	間違い探しで違う箇所を丸
hotfix	本番の問題を当てる緊急修正	水漏れに応急処置のテープ
コンパイル	実行できる形へ翻訳し組み上げる	レシピを作る手順書へ翻訳
メソッドの責務分離	役割ごとに処理を小さく分ける	接客・調理・会計を担当で分ける
シグネチャ	メソッドの呼び出し方の形	自販機の投入口と出る商品の決まり
DTO クラス	データをまとめて運ぶ入れ物	おかずを仕切って運ぶお弁当箱

スコープ	名前が通用する有効範囲	その部屋でだけ使える社員証
アクセス修飾子	誰まで使ってよいかの指定	関係者以外立入禁止の張り紙
ORDER_STATUS	注文が今どの段階かの状態	宅配の受付→配送中→お届け済み
enum	決めた選択肢だけ持てる型	信号の赤・黄・青だけ
findByCustomerId	顧客 ID で注文を探すメソッド	電話帳で名前から連絡先を引く
PaymentService	支払い処理を受け持つ担当	レストランの会計レジ係
マルチスレッド化	複数処理を同時並行で速くする	レジを増やして同時にさばく
ボトルネック	全体の速さを縛る詰まった箇所	瓶の細い首で流れが滞る

おわりに — 次の一歩

知らない言葉も、身近な例えとセットで覚えると、ぐっと頭に入ります。分からなくなったら、何度でも読み返してください。

◆ こんな進め方がおすすめ

- 「ひとことで言うと」だけ先に通して読む
- 身近な例えとセットで思い出す
- つながりをつたって、関係する用語をまとめて覚える

—— あなたのペースで大丈夫。応援しています! ——